NoteVision: Reconocimiento de Símbolos Musicales en Partituras Impresas Mediante una Red Convolutiva Recurrente con CTC

Roberto Serna Niño
A01709577
Inteligencia Artificial Avanzada — Prof. Benjamín Valdés Aguirre
Escuela de Ingeniería y Ciencias
Instituto Tecnológico y de Estudios Superiores de Monterrey, Campus Querétaro
Querétaro, México

Resumen—Este trabajo presenta NoteVision, un sistema de reconocimiento óptico de música (OMR) que convierte la imagen de un pentagrama impreso en una secuencia de símbolos musicales. Se utiliza el dataset PrIMuS (87,678 incipits monódicos) con etiquetas agnósticas (símbolo y posición en el pentagrama; vocabulario de 758 tokens) y particiones 80/10/10 sin fuga de datos. El modelo inicial (EXP-00) es una CRNN de 1,896,407 parámetros (cuatro bloques convolutivos, dos capas BiLSTM y una capa densa de 759 clases) entrenada con pérdida CTC en TensorFlow/Keras durante 8 épocas (3.94 h en una GPU T4). En el conjunto de validación completo (8,772 imágenes) obtiene un CER de 0.0067, una exactitud de secuencia de 0.8805 y un CER de forma de 0.0054, frente a ≈0.91 de un predictor trivial. La brecha respecto al entrenamiento es pequeña (CER 0.0057), por lo que no hay sobreajuste relevante. El análisis de errores muestra que el 74 % corresponde a omisiones (44 %) o inserciones (31 %) de símbolos pequeños y barras de compás, y solo el 17 % a errores de altura; además, unas 330 secuencias presentan una inconsistencia de etiquetas en la barra final. Con base en ello se plantea EXP-01, que reduce la compresión horizontal de $\times 8$ a $\times 4$. El conjunto de prueba se reserva para la comparación final.

Palabras clave—*Reconocimiento Óptico de Música, CRNN, CTC, Aprendizaje Profundo, PrIMuS, Tasa de Error de Caracteres, TensorFlow/Keras*

1. Introducción

Las partituras impresas existen como imágenes, pero no como información musical estructurada que un programa pueda manipular. El reconocimiento óptico de música (OMR, *Optical Music Recognition*) busca decodificar la notación musical a partir de imágenes [1]. Es un problema más difícil que el reconocimiento de texto: los símbolos se superponen con las líneas del pentagrama, su significado depende de la posición vertical y varios de ellos son pequeños y se dibujan pegados a otros (alteraciones, ligaduras, notas de adorno).

Una alternativa a las cadenas de procesamiento por etapas (binarización, segmentación, clasificación) es entrenar un único modelo de extremo a extremo. Las redes convolutivas recurrentes (CRNN) entrenadas con la pérdida CTC [5, 6] permiten leer una imagen de ancho variable y producir una secuencia de etiquetas sin necesidad de alinear cada símbolo con su posición horizontal; esta formulación ya se aplicó con éxito a partituras monofónicas [3, 1].

Este proyecto aborda una versión acotada del problema: una imagen recortada, impresa y de un solo pentagrama, y una salida secuencial. Sus aportaciones son: (i) un *pipeline* de datos con particiones sin fuga y preprocesamiento reproducible; (ii) una CRNN implementada en TensorFlow/Keras [13], con un protocolo de entrenamiento reanudable; (iii) un análisis de errores que separa los fallos del modelo de las inconsistencias de las etiquetas; y (iv) una propuesta de mejora (EXP-01) fundamentada en ese análisis.

Este reporte corresponde a la primera entrega: presenta el modelo inicial **EXP-00** evaluado sobre el conjunto de validación, y plantea la hipótesis de EXP-01 sin reportar aún sus resultados. El conjunto de prueba no se ha utilizado.

2. Descripción del Dataset

Se emplea PrIMuS (*Printed Images of Music Staves*) [4, 1], que contiene 87,678 incipits musicales reales, monódicos y de un solo pentagrama, extraídos de RISM, con las partituras renderizadas como imágenes y representaciones simbólicas codificadas como secuencias (Tabla 1). Existe una extensión, Camera-PrIMuS [2], con imágenes distorsionadas para simular fotografías; no se utiliza en este reporte. No se encontró una licencia explícita del dataset, por lo que se usa con fines académicos y con atribución; los datos no se redistribuyen y el repositorio documenta cómo descargarlos de la fuente original.

Cuadro 1: Resumen del dataset y de su representación

Aspecto	Valor
Imágenes	87,678 (PNG)
Origen	Incipits de RISM
Salida	Secuencia de tokens agnósticos
Vocabulario	758 tokens
Longitud máxima de etiqueta	75 tokens
Alto de entrada	128 px (fijo)
Ancho tras reescalar	361–3,417 px (validación)

Cada token combina la familia del símbolo y su posición en el pentagrama, por ejemplo `clef.G-L2`, `note.quarter-S2` o `barline-L1`, donde `L` indica una línea y `S` un espacio, numerados desde la parte inferior. Las familias principales son notas, silencios, claves, alteraciones, barras de compás, ligaduras, puntillos, cifras, notas de adorno, indicaciones de compás, silencios de varios compases y calderones. Esta re-

presentación separa la posición relativa en el pentagrama de la interpretación musical absoluta (altura real), que depende de la clave.
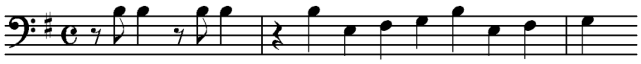


Figura 1: Ejemplo de entrada y de su etiqueta agnóstica. Primeros tokens: `clef.F-L4 accidental.sharp-L4 metersign.C-L3 rest.eighth-L3 note.eighth-S5 note.quarter-S5 rest.eighth-L3 note.eighth-S5 note.quarter-S5 barline-L1`. En el segundo compás la referencia etiqueta siete notas como `note.beamedRight0`, `Both0` y `Left0`, aunque son negras sueltas (Sección 4.5).

Los datos se dividieron 80/10/10 por grupo, de modo que las muestras relacionadas no se repartan entre particiones (Tabla 2).

Cuadro 2: Particiones (semilla 42)

Partición	Muestras	Proporción
Entrenamiento	70,138	80 %
Validación	8,772	10 %
Prueba	8,768	10 %

3. Metodología

3.1 Preprocesamiento y Representación de la Salida

Cada imagen se convierte a escala de grises, se redimensiona a un alto de 128 px conservando la proporción (interpolación bilineal con *antialias*) y se normaliza con tinta $= 1$ y fondo $= 0$. El ancho resultante es variable. Las etiquetas se codifican como identificadores enteros de tokens (0 a 757) con una longitud máxima de 75. Para la CTC se reserva la clase 0 como **blanco**; el token con identificador i corresponde a la clase $i+1$, de modo que el modelo produce 759 clases. El desplazamiento se aplica solo dentro de la pérdida y al decodificar.

3.2 Arquitectura CRNN

El modelo (Tabla 3, Fig. 2) tiene tres partes. Cuatro bloques convolutivos (convolución 3×3, normalización por lotes [9], ReLU y *max-pooling*) extraen características visuales y reducen la altura $\times16$ (de 128 a 8) y el ancho $\times8$. Para cada columna del mapa resultante se concatenan altura y canales ($8\times256=2048$), y una capa densa de 256 unidades con *dropout* [10] la proyecta. Dos capas LSTM bidireccionales [8] de 128 unidades por dirección modelan el contexto de izquierda a derecha y de derecha a izquierda; reciben una máscara con la longitud válida de cada imagen, de modo que el relleno se ignora. Una capa densa final de 759 unidades entrega los logits por fotograma. Cada fotograma corresponde a unos 8 px de la imagen reescalada.

Las capas BatchNorm añaden 960 parámetros no entrenables (estadísticas móviles), por lo que Keras reporta

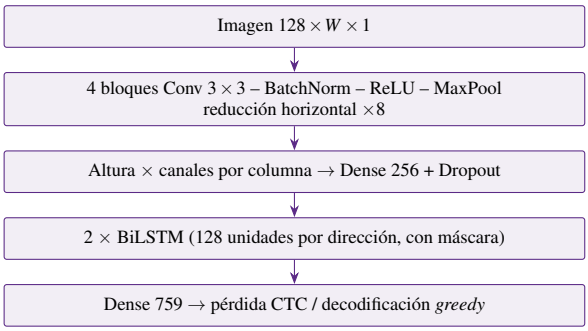


Figura 2: Arquitectura de la CRNN.

Cuadro 3: Parámetros entrenables por componente

Componente	Parámetros
4 convoluciones (32, 64, 128, 256 filtros)	387,360
BatchNorm (escala y desplazamiento)	960
Proyección (Dense 256)	524,544
2 BiLSTM (128 unidades)	788,480
Clasificador (Dense 759)	195,063
Total entrenable	**1,896,407**

1,897,367 en total. En *float32*, los pesos ocupan unos 7.6 MB, de modo que caben en un repositorio de código.

3.3 Pérdida CTC y Decodificación

La CTC [6, 7] suma la probabilidad de todos los alineamientos posibles entre los T fotogramas y la secuencia de referencia $\mathbf{y}$:

$$\mathscr{L}(\mathbf{x},\mathbf{y}) = -\log \sum_{\pi\in\mathscr{B}^{-1}(\mathbf{y})} \prod_{t=1}^{T} p(\pi_t \mid \mathbf{x}), \tag{1}$$

donde $\mathscr{B}$ fusiona repeticiones consecutivas y elimina el blanco. La pérdida de entrenamiento es la media de $\mathscr{L}$ sobre el lote. Como cada fotograma emite un solo símbolo, se requiere $T \geq |\mathbf{y}|$, y dos tokens idénticos consecutivos necesitan un blanco entre ellos. En inferencia se usa decodificación *greedy*: *argmax* por fotograma, fusión de repetidos y eliminación del blanco.

3.4 Entrenamiento

Se utiliza Adam [11] con recorte de gradiente por norma (5). La tasa de aprendizaje crece linealmente durante $W = 200$ pasos hasta $\eta_{\text{máx}} = 10^{-3}$ y después decae con una función coseno [12] hasta $\eta_{\text{mín}} = 10^{-5}$ al final de los N pasos:

$$\eta(t) = \eta_{\text{mín}} + \tfrac{1}{2}(\eta_{\text{máx}} - \eta_{\text{mín}})\big(1+\cos\pi p_t\big), \tag{2}$$

con $p_t = \text{mín}\{1, (t-W)/(N-W)\}$. Al ser función del paso, la reanudación tras una interrupción es directa.

Las imágenes tienen anchos muy distintos, por lo que los lotes se forman por cubetas de ancho: 21 cubetas (límites cada 100 px entre 600 y 2,200, más 2,600, 3,000 y 3,600), con

un presupuesto de 48,000 px por lote y un máximo de 64 imágenes (unas 41 en promedio). Cada imagen se rellena con ceros hasta el límite de su cubeta, lo que deja solo 21 formas de lote distintas. La configuración completa se resume en la Tabla 4.

Cuadro 4: Configuración de entrenamiento de EXP-00

Aspecto	Valor
Entorno	Google Colab, GPU T4
Software	TensorFlow 2.21.0, Keras 3.13.2
Optimizador	Adam, *clipnorm* 5
Tasa de aprendizaje	$10^{-3} \rightarrow 10^{-5}$, calentamiento de 200 pasos, coseno
Épocas	8 fijas (1,830 pasos por época; 14,640 en total)
Dropout	0.25
Precisión	*float32*
Semilla	42

3.5 Métricas

La métrica principal es la tasa de error de caracteres (CER), con tokens como unidad:

$$\mathrm{CER} = \frac{\sum_i d(\mathbf{r}_i, \mathbf{h}_i)}{\sum_i |\mathbf{r}_i|}, \tag{3}$$

donde d es la distancia de edición (inserciones, omisiones y sustituciones) entre la referencia $\mathbf{r}_i$ y la predicción $\mathbf{h}_i$. Además se reportan: la **exactitud de secuencia** (proporción de ejemplos con coincidencia exacta); el **CER de forma**, que compara solo la familia del símbolo e ignora la posición vertical y así separa errores de símbolo de errores de altura; y el **recall por familia**, calculado a partir del alineamiento. Como línea base se usan predictores triviales sin imagen (la secuencia más frecuente y la moda por posición), con CER ≈0.91 en validación.

3.6 Protocolo Experimental

Al final de cada época se calcula el CER en una submuestra fija de 2,000 imágenes de validación, común a todos los experimentos; el mejor modelo es el de menor CER en esa submuestra. El calendario fijo de 8 épocas es el criterio principal de parada, y la parada temprana (paciencia de 3 épocas desde la época 4) actúa solo como seguro. Se guardan puntos de control (pesos, estado del optimizador e historial) para reanudar tras desconexiones de Colab. Este protocolo es común a todos los experimentos para que las comparaciones sean limpias. La evaluación final usa el conjunto de validación completo; el conjunto de prueba se evalúa una sola vez, al elegir el modelo final.

4. Resultados

4.1 Entrenamiento de EXP-00

El entrenamiento duró 3.94 h (unos 29.5 min por época). La Tabla 5 y la Fig. 3 muestran la evolución sobre la submuestra de validación: el CER baja de 0.0317 a 0.0073 y la mejora se aplana en las últimas épocas, cuando la tasa de aprendizaje se acerca a su valor final. La parada temprana no se activó y el mejor modelo corresponde a la época 8.

Cuadro 5: EXP-00 por época (submuestra de validación de 2,000)

Época	Pérdida	CER	Exac.	CER forma
1	58.64	0.0317	0.592	0.0257
2	2.85	0.0156	0.760	0.0125
3	1.68	0.0107	0.815	0.0089
4	1.28	0.0089	0.845	0.0074
5	1.03	0.0082	0.853	0.0070
6	0.87	0.0077	0.861	0.0062
7	0.77	0.0075	0.866	0.0061
8	0.72	**0.0073**	0.870	0.0060

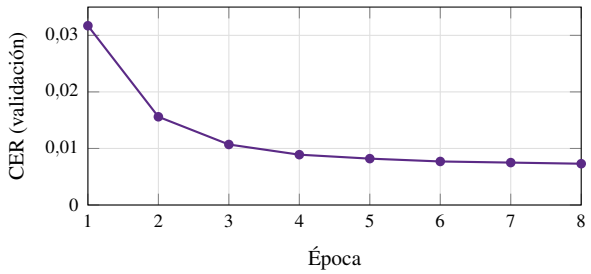


Figura 3: CER de EXP-00 por época en la submuestra de validación.

4.2 Desempeño en Validación

Con el mejor modelo se evaluó el conjunto de validación completo (Tabla 6). El CER es 0.0067 y el 88.05 % de las secuencias se reproduce sin ningún error, frente a un CER de ≈0.91 de los predictores triviales. El CER de forma (0.0054) es cercano al CER total, lo que indica que la mayor parte de los errores no se debe a la altura sino a la presencia o ausencia de símbolos.

Cuadro 6: Métricas de EXP-00

Conjunto	CER	Exac.	CER forma
Predictor trivial (val.)	≈0.91	—	—
Entrenamiento (2,000)	0.0057	0.8995	0.0044
Validación (2,000)	0.0073	0.870	0.0060
Validación completa (8,772)	0.0067	0.8805	0.0054

4.3 Análisis de Sesgo, Varianza y Ajuste del Modelo

Para diagnosticar el ajuste se evaluó, en modo de inferencia (sin *dropout*), una muestra aleatoria de 2,000 imágenes de

entrenamiento y se comparó con el conjunto de validación completo (Tabla 6).

Varianza: baja. La brecha es pequeña: el CER pasa de 0.0057 en entrenamiento a 0.0067 en validación (diferencia relativa del 15 %), y la exactitud de secuencia de 0.8995 a 0.8805 (menos de dos puntos porcentuales). El modelo no memoriza el conjunto de entrenamiento.

Sesgo: bajo en términos absolutos, pero no nulo. El modelo sigue cometiendo errores sobre las propias imágenes de entrenamiento (el 10 % de esas secuencias no coincide exactamente). Parte de este error residual no es reducible, porque proviene de inconsistencias de las etiquetas (Sección 4.5), y otra parte corresponde a símbolos pequeños que el modelo aún no resuelve (Sección 4.4).

Nivel de ajuste: adecuado, sin sobreajuste. Dado que no hay sobreajuste, técnicas de regularización o aumento de datos no abordarían el error dominante en esta distribución de datos, lo que orienta la hipótesis de EXP-01 hacia la resolución del modelo y no hacia la regularización.

4.4 Análisis de Errores

En validación hay 1,604 errores sobre 240,867 tokens, distribuidos en 1,048 secuencias con al menos un error. La Tabla 7 y la Fig. 4 los descomponen por tipo.

Cuadro 7: Descomposición del CER por tipo de error

Tipo	Conteo	Tasa	Parte
Omitidos (borrado)	700	0.0029	43.6 %
Insertados	493	0.0020	30.7 %
Mismo símbolo, altura incorrecta	266	0.0011	16.6 %
Símbolo incorrecto	145	0.0006	9.0 %
Total	**1,604**	**0.0067**	100 %

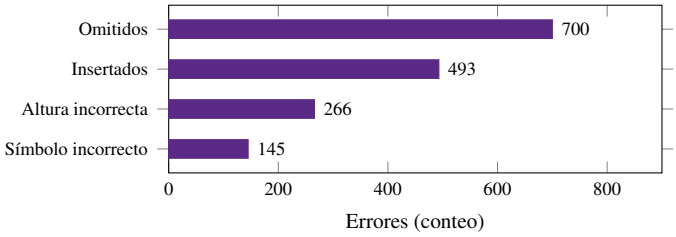


Figura 4: Errores de EXP-00 en validación por tipo.

Errores de altura. Los 266 casos de altura incorrecta son errores pequeños: el 62 % se desplaza un paso del pentagrama y el 34 % dos pasos (Tabla 8). El 81 % (215) corresponde a notas de adorno (`gracenote`), que son pequeñas; las confusiones más frecuentes son, por ejemplo, `gracenote.sixteenth-L4→S3` (15 casos) y `L4→S4` (10 casos).

Omisiones. Predominan los símbolos pequeños (Tabla 9). Las alteraciones suman 471 omisiones (29 % de todos los errores): becuadros (336), bemoles (110) y sostenidos (25).

Cuadro 8: Desplazamiento en pasos del pentagrama (predicción − referencia) en los errores de altura

Desplazamiento	Casos	Parte
±1	165	62.0 %
±2	91	34.2 %
Otros (3 a 6)	10	3.8 %

Los becuadros tienen un *recall* de 0.768 (1,489 apariciones), el más bajo entre las familias frecuentes. Se omiten además 116 barras de compás y 77 ligaduras.

Inserciones. Las inserciones más frecuentes son 216 barras de compás `barline-L1` y, a continuación, ligaduras `slur.start` y `slur.end` en distintas posiciones.

Cuadro 9: *Recall* por tipo de símbolo en validación (el *recall* de forma ignora la posición)

Tipo	Soporte	Recall	R. forma	Omit.
Nota	133,595	0.9991	0.9994	6
Barra de compás	32,594	0.9963	0.9963	116
Alteración	17,331	0.9710	0.9717	471
Silencio	13,114	0.9998	0.9998	1
Puntillo	11,826	0.9994	0.9997	2
Cifra	11,175	0.9992	0.9992	1
Clave	8,849	0.9994	0.9994	3
Indic. de compás	4,796	1.0000	1.0000	0
Ligadura	2,594	0.9680	0.9680	77
Silencio multicompás	2,294	0.9996	0.9996	0
Nota de adorno	2,257	0.8848	0.9801	22
Calderón	442	0.9977	0.9977	1

Por familia, las de menor *recall* con al menos 100 apariciones son `gracenote.sixteenth` (0.747, pero 0.967 si se ignora la posición), `accidental.natural` (0.768), `slur.end` (0.965) y `slur.start` (0.972). Las notas de duración común (negras, corcheas, blancas y notas ligadas por haz) superan 0.999.

Longitud y ancho. El CER crece con la longitud de la secuencia: 0.0075 hasta 20 tokens, 0.0063 entre 21 y 40, y 0.0105 entre 41 y 60 (329 secuencias; Tabla 10). El efecto del ancho es leve, salvo en el quintil de imágenes más anchas, donde la exactitud de secuencia baja a 0.8414.

4.5 Inconsistencias en las Etiquetas

La inspección de ejemplos mostró dos peculiaridades de las referencias que no son errores del modelo.

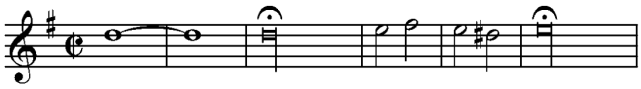
Barra final. En 219 secuencias con error, la predicción termina con una `barline-L1` que la referencia no incluye, y en 111 ocurre lo contrario. Solo el 52.3 % de las referencias de validación termina con barra. En los dos ejemplos revisados con una barra extra al final, la barra **sí está dibujada** en la imagen (Fig. 5a), es decir, el modelo acierta y la referencia la omite. Estas 330 secuencias equivalen a cerca de un tercio de las 1,048 secuencias con error y, como máximo, a ≈20 % de los errores a nivel de token. Si se excluye-

Cuadro 10: CER y exactitud por longitud de referencia y ancho de imagen
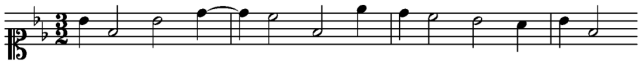
Grupo	Muestras	CER	Exac.
Tokens de referencia			
1–20	1,296	0.0075	0.9128
21–40	7,145	0.0063	0.8810
41–60	329	0.0105	0.7447
61–75	2	0.0163	0.5000
Ancho tras reescalar (quintiles, px)			
361–918	1,764	0.0071	0.8883
919–1,072	1,752	0.0063	0.8864
1,073–1,219	1,750	0.0057	0.9006
1,220–1,433	1,753	0.0064	0.8859
1,434–3,417	1,753	0.0078	0.8414

ran, el CER sería del orden de 0.0053 (una cota aproximada: $(1{,}604-330)/240{,}867$).

Haces de longitud cero. Algunas notas se etiquetan como `note.beamedRight0`, `Both0` o `Left0` (haz sin barras), pero en las imágenes revisadas son negras sueltas (Fig. 1); el modelo predice `note.quarter`, que es lo visible.



(a) Barra final dibujada; la referencia no la incluye y el modelo la predice.



(b) Ligadura evidente (entre la cuarta y la quinta nota) que el modelo omite.

Figura 5: Ejemplos de errores de EXP-00.

Por el contrario, los errores del tipo (b) sí son del modelo. Un segundo caso de ligadura omitida corresponde a una ligadura casi superpuesta con la quinta línea del pentagrama, visualmente ambigua.

4.6 Hipótesis para EXP-01

El análisis anterior sugiere que el límite actual no es la capacidad de generalizar sino la resolución con que el modelo ve los símbolos pequeños y adyacentes (becuadros, bemoles, ligaduras, notas de adorno). Con una reducción horizontal de $\times 8$, cada fotograma de la CTC cubre unos 8 px y solo puede emitir un símbolo, por lo que varios símbolos próximos compiten por el mismo fotograma. Este mecanismo es una hipótesis: el análisis solo muestra que esos símbolos son la principal fuente de omisiones.

EXP-01 propone un único cambio respecto a EXP-00: reducir la compresión horizontal de $\times 8$ a $\times 4$ (la tercera capa de *pooling* pasa de 2×2 a 2×1 y la longitud lógica de $\lfloor W/8 \rfloor$ a $\lfloor W/4 \rfloor$). Los parámetros no cambian (1,896,407) y todo lo demás se mantiene: datos, particiones, semilla, *pipeline*, optimizador, calendario, 8 épocas y submuestra de validación. El criterio de éxito, sobre el conjunto de validación completo, es:

- un CER al menos 10 % menor (de 0.0067 a 0.0060 o menos);
- mayor *recall* de becuadros (0.768) y de ligaduras, y menos omisiones;
- como métrica secundaria, el CER ignorando una barra final, que neutraliza la inconsistencia de etiquetas.

Cada experimento es una sola corrida, por lo que no se mide la variación entre corridas; de ahí el umbral del 10 %. Los resultados de EXP-01 no forman parte de este reporte.

5. Discusión

El modelo inicial resuelve la tarea con muy poco error: un CER de 0.0067 en validación, frente a $\approx$0.91 de la línea base trivial, con el 88 % de los pentagramas transcritos sin error. Como referencia, Calvo-Zaragoza et al. [3] reportaron errores cercanos al 2 % por símbolo sobre 90,000 partituras sintéticas con más de 50 etiquetas; los valores no son directamente comparables, porque este trabajo usa otro conjunto de etiquetas (758 tokens con posición), otras particiones y otra métrica.

El hallazgo central es que los errores restantes son de otro tipo del que cabría esperar. La altura de los símbolos rara vez falla, y cuando falla lo hace por uno o dos pasos; en cambio, el 74 % de los errores son omisiones o inserciones, concentradas en símbolos pequeños o dibujados junto a otros (becuadros, bemoles, ligaduras, notas de adorno). Esto es coherente con la restricción de la CTC de un símbolo por fotograma, aunque la causa exacta requiere el experimento EXP-01 para confirmarse.

Una parte del error no es atribuible al modelo. La inconsistencia de la barra final afecta a unas 330 secuencias, y las etiquetas `beamed...0` no corresponden a lo visible. Esto establece un piso para el CER con estas etiquetas y justifica reportar, además del CER, una métrica que neutralice la barra final. También implica que una mejora grande del CER total no debe esperarse, porque parte del error es de las etiquetas.

La brecha pequeña entre entrenamiento y validación indica que el modelo tiene margen para ser más expresivo antes de que la regularización sea necesaria. Por eso la primera mejora que se prueba modifica la resolución y no la regularización o el aumento de datos, que serían más relevantes para imágenes con distorsiones como las de Camera-PrIMuS [2].

Limitaciones. (i) Los resultados provienen de imágenes sintéticas y limpias; el desempeño en fotografías o partituras escaneadas no está garantizado. (ii) Cada experimento es una sola corrida y no se mide la varianza entre semillas. (iii) El mejor modelo se selecciona con una submuestra de 2,000 imágenes que forma parte del conjunto de validación completo, por lo que la cifra de validación completa no es totalmente independiente de la selección; el conjunto de prueba, que sí

lo es, no se ha utilizado. (iv) El alcance se limita a un pentagrama monofónico impreso; las partituras polifónicas, los PDF multipágina y las manuscritas quedan fuera. (v) El modelo predice posiciones relativas; la altura absoluta depende de la clave y de la armadura.

Interfaz de predicción. El repositorio incluye un programa de consola autocontenido (`predict.py`) que carga los pesos y el vocabulario, aplica el mismo preprocesamiento del entrenamiento y entrega la secuencia de tokens para una imagen PNG o JPG. Si no se predice ninguna clave, el programa asume la clave de sol y lo marca explícitamente como supuesto, nunca como detección.

Como proyecto complementario, NoteVision-Web (https://github.com/Rigby119/NoteVision-Web) usa estos mismos pesos para mostrar el resultado como notación y audio sintetizado.

6. Conclusiones

- Se construyó un *pipeline* completo de reconocimiento de símbolos musicales: preprocesamiento reproducible, particiones 80/10/10 sin fuga de datos, una CRNN de 1,896,407 parámetros en TensorFlow/Keras entrenada con CTC y un protocolo de entrenamiento reanudable.
- EXP-00 alcanzó un CER de 0.0067, una exactitud de secuencia de 0.8805 y un CER de forma de 0.0054 en el conjunto de validación completo (8,772 imágenes), frente a $\approx$0.91 de un predictor trivial.
- El diagnóstico de sesgo y varianza mostró una brecha pequeña entre entrenamiento (CER 0.0057) y validación (0.0067): no hay sobreajuste y la regularización no es la prioridad.
- El 74 % de los errores son omisiones o inserciones, sobre todo de becuadros, bemoles, ligaduras y notas de adorno; los errores de altura (17 %) son casi siempre de uno o dos pasos.
- Las etiquetas de PrIMuS presentan inconsistencias (barra final en unas 330 secuencias; haces de longitud cero) que limitan el CER alcanzable y deben considerarse al interpretar los resultados.
- Se plantea EXP-01, con un único cambio (reducción horizontal de $\times 8$ a $\times 4$), un criterio de éxito definido de antemano y comparación sobre las mismas particiones y métricas.
- Trabajo futuro: ejecutar EXP-01 y compararlo con EXP-00; evaluar el conjunto de prueba una sola vez con el modelo final; probar con Camera-PrIMuS; y ampliar a otros tipos de partituras (lectura automática de PDF multipágina y partituras manuscritas).

Referencias

[1] J. Calvo-Zaragoza and D. Rizo, "End-to-End Neural Optical Music Recognition of Monophonic Scores," *Applied Sciences*, vol. 8, no. 4, art. 606, 2018. [En línea]. Disponible: https://doi.org/10.3390/app8040606

[2] J. Calvo-Zaragoza and D. Rizo, "Camera-PrIMuS: Neural End-to-End Optical Music Recognition on Realistic Monophonic Scores," in *Proc. 19th Int. Society for Music Information Retrieval Conf. (ISMIR)*, Paris, France, 2018, pp. 248–255. [En línea]. Disponible: https://ismir2018.ircam.fr/doc/pdfs/33_Paper.pdf

[3] J. Calvo-Zaragoza, J. J. Valero-Mas, and A. Pertusa, "End-to-End Optical Music Recognition Using Neural Networks," in *Proc. 18th Int. Society for Music Information Retrieval Conf. (ISMIR)*, Suzhou, China, 2017, pp. 472–477.

[4] Pattern Recognition and Artificial Intelligence Group, "The Printed Images of Music Staves (PrIMuS) dataset," Universidad de Alicante. [En línea]. Disponible: https://grfia.dlsi.ua.es/primus/

[5] B. Shi, X. Bai, and C. Yao, "An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[6] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," in *Proc. 23rd Int. Conf. on Machine Learning (ICML)*, 2006, pp. 369–376.

[7] A. Hannun, "Sequence Modeling with CTC," *Distill*, 2017. [En línea]. Disponible: https://distill.pub/2017/ctc

[8] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[9] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. 32nd Int. Conf. on Machine Learning (ICML)*, 2015, pp. 448–456.

[10] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[11] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. 3rd Int. Conf. on Learning Representations (ICLR)*, 2015.

[12] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in *Proc. 5th Int. Conf. on Learning Representations (ICLR)*, 2017.

[13] M. Abadi *et al.*, "TensorFlow: A System for Large-Scale Machine Learning," in *Proc. 12th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.